

TECHNICAL CASE STUDY

Docling PDF Parsing & Layout Challenges

An architectural analysis of layout misclassification, reading-order fragmentation in borderless tables, and programmatic DOM reordering solutions.

SECTION 01

Problem Diagnosis

Understanding why standard heuristic parsing breaks on complex multi-column document layouts.

The Core Processing Challenge

Target Logical Goal

Preserve true semantic hierarchy and linear reading flow across side-by-side comparison blocks, ensuring sections remain adjacent in downstream Markdown and RAG contexts.

Observed Failure Mode

The parser processes the left column downward through subsequent sections, pushing parallel right-column content ("Analytics Layer") to the end of the document page.

Root Cause Breakdown

- ✔ **Layout Model Misclassification:** DocLayNet tags borderless side-by-side columns as standard body text instead of table bounding boxes.
- ✔ **Vertical Graph Traversal:** Spatial reading-order heuristics traverse left-column boxes straight down through full-width tables.
- ✔ **Asymmetric Caption Anchors:** Unilateral captions (e.g., Figure 1) break structural symmetry, bypassing TableFormer detection.

Page header

Title

Section heading

Paragraph

Subsection heading

Selection marks

Table

Table caption

Figure caption

Figure

Page number

Page footer

This is the header of the document.

This is title

1. Text
Latin refers to an ancient Italic language originating in the region of Latium in ancient Rome.

2. Page Objects
2.1 Table
Here's a sample table below, designed to be simple for easy understand and quick reference.

Name	Corp	Remark
Foo		
Bar	Microsoft	Dummy

Table 1: This is a dummy table

2.2. Figure
Figure 1: Here is a figure with text

3. Others
AI Document Intelligence is an AI service that applies advanced machine learning to extract text, key-value pairs, tables, and structures from documents automatically and accurately:
☒ clear
☒ precise
☐ vague
☒ coherent
☐ Incomprehensible
Turn documents into usable data and shift your focus to acting on information rather than compiling it. Start with prebuilt models or create custom models tailored to your documents both on premises and in the cloud with the AI Document Intelligence studio or SDK.
Learn how to accelerate your business processes by automating text extraction with AI Document Intelligence. This webinar features hands-on demos for key use cases such as document processing, knowledge mining, and industry-specific AI model customization.

1 | Page

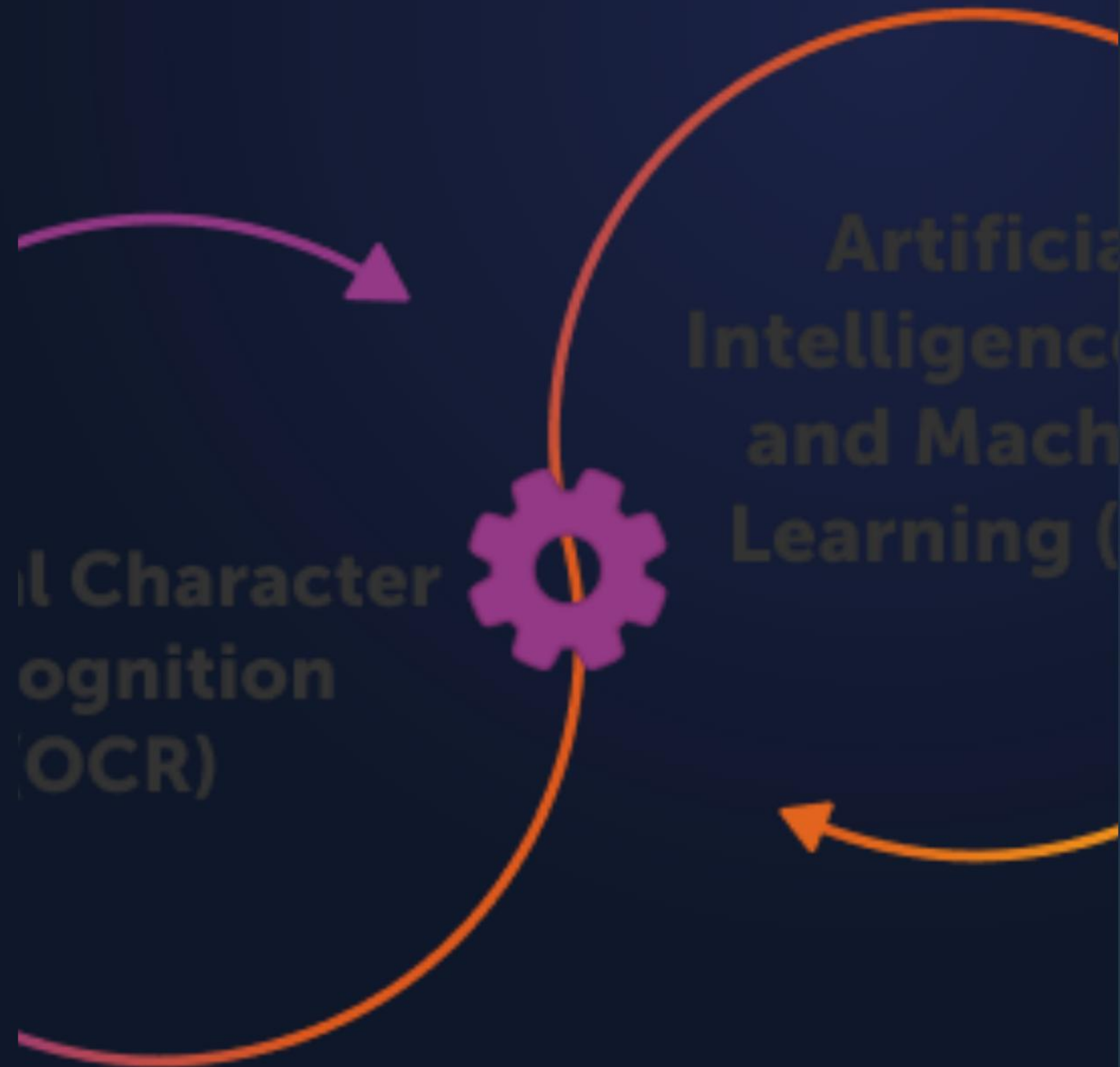
Benchmark Document Dynamics

Edge-Case Benchmark

The sample PDF (Stage 1.4.2.5.3) is engineered specifically to evaluate document intelligence resilience under non-standard visual scanning paths.

Structural Asymmetry

Page 2 features symmetric grid lines and parses cleanly into tables, whereas Page 1 mixes floating captions with borderless columns, triggering parser traps.



Why Native Pipelines Bypassed It



Default Pipeline

Prioritizes speed by skipping heavy cell-matching on borderless regions, falling back to basic top-to-bottom bounding box graphs.



TableFormer Bypass

Even in ACCURATE mode, TableFormer is only invoked if the upstream DocLayNet model first generates a table bounding box.



Graph Disruption

Floating figures beneath the left column attract vertical edge connections, severing the horizontal link to adjacent right blocks.

Text Extraction Fidelity

100%

Text Recovery Rate

Complete Content Retention

Docling never lost or dropped the missing "Analytics Layer" text. Every token, character span, and bounding box was stored safely inside the JSON DOM model (`doc.texts`). The issue was purely a reading-order sequence bug, making it 100% recoverable.

SECTION 02

Resolution Strategies

Implementing spatial DOM reordering and pipeline enhancements for production RAG.

Available Mitigation Pathways



Spatial DOM Sorting

Resolve JSON pointers, group coordinates into vertical Y-bands, and order horizontal blocks left-to-right prior to Markdown generation.



Vision–Language Models

Switch to VLM pipelines (Granite-Doc or Qwen2.5-VL) that process visual 2D space holistically rather than bounding box trees.



Upstream Layout Fixes

When controlling source documents, wrap parallel comparison blocks inside explicit table borders or aligned flex grids.

Solution Matrix Comparison

Mitigation Approach	Implementation Effort	Accuracy	Resource Overhead
TableFormer ACCURATE Mode	Low (Config switch)	Moderate	Low (CPU/GPU)
Python Spatial DOM Sorting	Low (Post-processing script)	High	Minimal (<10ms)
VLM Pipeline (e.g., Qwen2.5-VL)	Medium (Model swap)	Very High	High (GPU required)

RAG Ingestion Workflow



1. Docling Parse

Extract DOM, bounding boxes, tables, and images.



2. DOM Reorder

Apply spatial Y-band sorting to fix parallel columns.



3. Hybrid Chunk

Tokenize preserving headers & table integrity.



4. Vector Store

Embed with BGE-M3 and persist into ChromaDB.

Questions & Discussion

Docling Ingestion & Multimodal RAG Pipeline Architecture

Data Ingestion Stage | Open-Source Multimodal RAG Roadmap

Image Sources



<https://learn.microsoft.com/en-us/azure/ai-services/document-intelligence/media/document-layout-example-new.png?view=doc-intel-4.0.0>

Source: learn.microsoft.com



https://www.automationanywhere.com/sites/default/files/images/rpa-seo-template/idp/idp-updated-banner_en.png

Source: www.automationanywhere.com